

Evaluation of Autoencoders-based strategies for dimensionality reduction of high-dimensional financial data

Hello! We are presenting our thesis outline and plan for the topic “Evaluation of Autoencoders-based strategies for dimensionality reduction of high-dimensional financial data”.

=/=

Our main goal of that thesis would be to investigate how different Autoencoder-based models perform in dimensionality reduction tasks, in particular for financial data. The baseline model that we will compare AE to is PCA, as it is the most widely used dimensionality reduction technique out there.

Also we would like to analyze how latent representations created by these models can be interpreted and used, which insights we can extract from them.

=/=

Here is the list of models we are going to use.

1. Regular PCA - as a baseline. We are going to compare our results with that.
2. Regular Autoencoder
3. Variational Autoencoder.
4. Beta-VAE
5. VAE with t-student posterior assumption. Regular VAE uses the Gaussian assumption and our data is not Gaussian, so the t-student might give better results

Also if we have time, we might look into the copula-based VAE, which should perform really well in that domain, as it is stated in papers that propose them. Especially Vine Copula VAE.

=/=

So, what is an Autoencoder? It is a neural network architecture with a bottleneck in the middle. The idea behind it is simple - we are trying to learn an encoder that compresses our data into a low-dimensional embedding and a decoder that decompresses it back with hopefully a small reconstruction error. After such a model is trained, the latent representation of the original data should have the same information as the original but have a much lower dimension.

There are two key differences from the PCA - first of all it is a non-deterministic approach. We need to train a Neural Network, which can lead to better or worse performance.

Secondly, as it is a neural network, we can use non-linear activation functions leading to non-linear transformations. PCA is a linear transformation only.

=/=

Now about the Variational autoencoder. Again it is a neural network that has a bottleneck layer, but now we are not mapping our original data to a single point here, but we are mapping it to a distribution in that latent dimension. Original VAE uses Gaussian distributions, so each point is encoded into Gaussian parameters - Mean and std.dev. Then we sample from that distribution and decode the sampled value. It might sound like a very weird thing to do and that is right. We are trying to extract as much useful information from our original data as possible, but for some reason we also add sampling noise. The advantage of that method lies in the fact that now we can model the latent distribution really well and understand its internal dependencies and structure. We will show an example a couple of slides later.

=/=

So how do we train such a model? Initially in the same way as the basic Neural Network but we need to use a special kind of loss function. It consists of two parts:

1. The first one is the reconstruction loss - MSE or Cross-Entropy. Essentially it depends on our posterior assumption, basic VAE uses Gaussian posterior so we will use Gaussian distribution here.
2. Second term is the KL divergence of our resulting distribution of latent space and the prior distribution. Again, the prior can be modelled with different distributions, but the original VAE uses standard gaussian prior so we will use that one. The idea behind that term is that it "pushes" our latent distribution to some boundaries as otherwise the model would collapse to some random unstructured mapping (almost like original autoencoder) and the whole idea of being able to analyze the distribution of the latent space will be lost.

=/=

So now back to the latent space - here we have an example of the latent space for MNIST dataset - not quite a financial dataset but it gives a great understanding of what is happening. Here images of handwritten digits are mapped to a 2D latent space, different colors represent different digits. We clearly see how each digit forms an ellipsoid (or something close to ellipsoid), and similar digits are near each other. So 0 is near 6 etc. We can see the distribution of each of the digits - that 0 has a much higher variability on how that is written, compared to 2 and so on. We can use the same logic for our dataset and map different dates to a lower dimensional space and see if we can find some patterns there - most probably days of high return would be far away from days of negative returns. Maybe even some market regimes could be found as separate clusters.

=/=

Also there is an interesting thing as convex interpolation in the latent space, but it is used for generation of new samples, and hardly can be applied in our domain. But that is just a

beautiful thing that we wanted to show you as well. We can generate gradual transitions between samples - as here we transform a 0 into a 6.

=/=

We also had Beta-VAE and t-student VAE in our architecture outline - they have the same structure as a regular VAE but have different losses.

Beta-VAE adds a scaling parameter for the KL divergence term. Now we can change how much influence the KL term has in our model, and most importantly it can be changed for different training stages. So in the beginning it can be very small in order to let the model learn the data at least somehow, and then it can be gradually increased, making the model change its latent representation to fit our prior.

Another approach is to change prior or posterior assumptions. So in regular VAE we assumed that distributions are Gaussian, and in our case it is not, so we can change the assumption to t-Student distribution. If we change the posterior assumption - it would influence the reconstruction loss term, t-Student would penalize errors in the tails less, making the model more stable. And using t-Student as a prior would change the KL divergence term, allowing the latent representations to have more t-student-like distributions.

=/=

As for the data we took S&P 500 Stock Data from Kaggle. It contains the data for all current S&P 500 companies. The data is available for 5 consecutive years from 2013 for 505 different stocks. There are stocks with missing values because the corresponding companies were not a part of S&P 500 index. We decided to keep the ones for which the whole timeline was available which resulted in 470 stocks.

=/=

For the visualization we took 4 stocks from different sectors: Apple for Technology, Amazon for Consumer Discretionary, Bank of America for Financials and American Electric Power for Utilities. As we can see raw prices are not a stationary process, so we will take log returns as it is best practice when dealing with such data. However, they are also not from the Normal distribution - they have heavier tails.

=/=

For the heatmap we took 10 stocks from each of the 4 sectors and plotted their return correlation. As can be seen the stocks in the same sector have a higher correlation in log returns, especially the Financials and Utilities sectors. Almost all returns are correlated with each other but the correlation is not too high (the mean Pearson correlation coefficient is just 0.28).

=/=

Methodology.

We are going to test our models in a standard time-series way. We will split our data into the following splits, so that train data does not leak into the test.

Main metrics that we will use for the dimensionality reduction comparison are Preserved variance, reconstruction error and entropy of the residuals.

==

Right now we have run tests on PCA and regular AE.

First of all we tested different activation functions - sigmoid, relu and tanh. We can see that sigmoid performs much worse than others because of the vanishing gradients problem. Relu and tanh perform much better.

==

We've run the same tests for different model architectures and we can see that they are very close to each other, but for large k (32) the smallest model slightly outperforms its competitors.

==

Now comparing the best AE model with the PCA we can see that they perform really close to each other but PCA is still better. On one hand that is an obvious result - PCA by definition is the most optimal linear dimensionality reduction technique, but on the other hand AE is not restricted by linearity. Further analysis of the results would be performed in our thesis.

==

So as a conclusion we can say that AE slightly underperforms compared to PCA, but its latent dimension can have some insights like internal correlations, which may lead to not the most efficient dimensionality reduction.

Also from theory VAE should perform worse than regular AE in terms of reconstruction, but we should be able to extract more information from the latent distributions than from regular AE.

==

Here is the list of the main literature we used. In reality we used more, and we will add a full list in the paper itself.

==

Thank you everyone. We would be pleased to hear your questions or comments.